Maximum Diversity Problem Report

Carlos Gómez Sáez & Raúl Rodríguez Gómez

This study compares a baseline GRASP with a GRASP combined with Tabu Search for the Maximum Diversity Problem. Under equal time budgets, the more complex method does not outperform the simpler one on average.

The comparison clarifies which implementation choices kept the evaluation fair, which programming difficulties shaped the final code, and which operating regimes favor each method.

1. Project Goals and Problem Context

The Maximum Diversity Problem asks for exactly p elements from a set of n candidates so that the total pairwise distance inside the selected subset becomes as large as possible.

Problem formulation.

maximize sum_{i<j} d_{ij} x_i x_j

subject to sum_i x_i = p, x_i in {0,1} for all i.

We use MDG benchmark instances that provide undirected pairwise distances for problems of sizes n = 100 and n = 500.

Two practical questions shaped the project from the start. The first one concerned the construction policy in each algorithm, especially the role of the alpha parameter in GRASP. The second one asked whether Tabu Search really improves the basic GRASP when both methods run under the same wall-clock budget. Those two questions shaped the code and the experimental pipeline because they forced a clean separation between construction, improvement, calibration, and final comparison.

The value of alpha could not stand alone. In our implementation, alpha controls the width of the restricted candidate list during construction. A low alpha pushes the method toward greedier choices, while a high alpha pushes it toward exploration. That design choice matters because the quality of the starting solution strongly affects local improvement, and our main conjecture from the beginning was that plain GRASP and GRASP plus Tabu Search would react differently to that trade-off.

Calibration therefore became part of the experiment itself instead of a cosmetic tuning step. An arbitrary alpha or tabu tenure could make one method look weaker than it really is, so the project needed a parameter search before any final conclusion made sense.

Path Relinking was considered as an initial extension, but the final implementation chose Tabu Search because it integrated more directly with the existing GRASP structure and made the intensification mechanism easier to isolate experimentally. In practice, the Tabu List offered a clean way to study how short-term memory can escape local optima without rewriting the constructive and local-search backbone of the original code.

2. Implementation Workflow

The implementation follows a simple workflow: load each instance into a symmetric distance matrix, construct candidate solutions with GRASP, improve them with either best-improvement

local search or Tabu Search, calibrate the key parameters on a representative subset, and compare both methods under matched time budgets.

3. Key Implementation Decisions

3.1 Algorithms: succinct description and pseudocode

GRASP (baseline) - pseudocode:

- INPUT: instance, p, alpha (or alpha = -1 meaning random-alpha), TIME_LIMIT

- best := None; start = wall-clock();

- while wall-clock() - start < TIME_LIMIT:

  - if alpha == -1: sample alpha from candidate set

  - s := ConstructGreedyRandomized(instance, p, alpha)

  - s := BestImprovementLocalSearch(s) // exhaustive one-out one-in swap

  - if s.of > best.of: best := s

- return best

GRASP+Tabu Search - pseudocode:

- INPUT: instance, p, alpha, tabu_tenure, TIME_LIMIT

- best := None; start = wall-clock();

- while wall-clock() - start < TIME_LIMIT:

  - s := ConstructGreedyRandomized(instance, p, alpha)

  - s := BestImprovementLocalSearch(s)

  - s := TabuSearch(s, tabu_tenure, remaining_time = TIME_LIMIT - (wall-clock()-start))

  - if s.of > best.of: best := s

- return best

TabuSearch (brief behavior):

- Use one-out/one-in swap neighborhood.

- Maintain FIFO tabu_list of recently removed nodes; block moves that re-introduce a tabu node unless aspiration (improve global best) allows it.

- Respect an inner time bound passed from the outer wrapper; check wall-clock inside the main loop.

3.2 Solution representation and incremental evaluation

Solution representation: Each solution is a dict with fields: `sol` (set of selected node indices), `of` (current objective), and `instance` (pointer to instance data). Membership tests use a Python `set` for O(1) contains checks.

Incremental evaluation: Implement `distanceToSol(sol, v)` and incremental add/remove operations that update `sol['of']` in O(n_selected) per update rather than recomputing the full objective.

3.3 GRASP construction and alpha control

The constructive phase starts from a random first element and then grows the partial solution through a restricted candidate list. Each candidate receives a score based on its current marginal contribution, and alpha controls the width of that list. Our implementation also keeps the special value alpha = -1 so that each construction can sample a random alpha instead of using a fixed value.

That design kept the construction policy explicit. When the calibration stage later changed alpha, the measured effect really came from the width of the restricted candidate list and not from some hidden source of randomness elsewhere in the code.

3.4 Best-improvement local search

Local search follows exhaustive best improvement over the full one-out one-in swap neighborhood. The project deliberately avoided a lighter first-improvement variant because the baseline needed to stand as a serious benchmark instead of an intentionally weak version of GRASP. That choice makes each restart more expensive, but it also guarantees that every completed run stops at a genuine local optimum with respect to the swap neighborhood.

3.5 Tabu Search and memory

The Tabu Search phase uses the same one-out one-in swap neighborhood, but the movement rule changes through a FIFO tabu list and a basic aspiration criterion.

Tabu list concrete note: The final code uses a FIFO Python `list` as `tabu_list`; membership tests are `if v_in in tabu_list` (O(tenure)). Because selected tenure values are small (<=30) and tenure is constant per run, this design keeps code simple and fast in practice. For the large instances, tenure = 10 became the engineering trade-off between cheap membership checks, readability, and search behavior.

The Tabu Search implementation uses a FIFO Python list and the membership check `if v_in in tabu_list`.

3.6 Global time budgeting and reproducibility

Both final algorithms operate under a global wall-clock budget. The GRASP baseline repeats construction plus best-improvement local search while time remains, and the GRASP plus Tabu Search variant forwards the remaining budget into each Tabu Search call. Because the time checks happen at iteration boundaries, both wrappers can show slight natural overruns, but the design still keeps the comparison fair in actual runtime.

Calibration outputs are reused automatically in the final comparison, which avoids manual parameter transcription and keeps the experimental pipeline consistent.

The runtime coupling with Tabu Search is handled in algorithms/grasp_ts.py. Each restart measures the remaining global budget and passes that value into tabu_search.improve(...), so the inner while-loop works with the same wall-clock constraint as the outer GRASP wrapper instead of with an unrelated local counter.

These data-structure choices are not cosmetic. They directly determine how many restarts and neighborhood scans fit under the same time budget, so they belong to the experimental design as much as they belong to the implementation.

4. Difficulties We Encountered During Programming and Experimentation

Instance loading created one of the first real programming difficulties. Each input file provides distances for undirected pairs, so the loader had to mirror every value from d[u][v] into d[v][u] immediately. A small asymmetry at that point would corrupt construction and swap evaluation in a way that later debugging would barely detect. Concentrating the parsing logic in a single module reduced that risk.

That bug actually appeared during the first implementation attempts. Some objective values looked inconsistent across repeated runs, and the first suspicion fell on local search. A closer inspection pointed somewhere else: the distance structure itself. Once the code checked matrix symmetry systematically, the rest of the behavior became much more stable and much easier to debug.

Runtime control created a second difficulty. The baseline GRASP and the GRASP plus Tabu Search variant do not spend time in the same way. Tabu Search can burn much more effort inside one restart, so a naive local budget would have made the comparison unfair. The final design treats time as a global resource and passes the remaining budget into the Tabu Search phase. That solution required more care than a fixed-loop implementation, but it let the project answer the professor's question honestly.

Calibration raised a third difficulty. A full grid over all alpha values and all tenure values for GRASP plus Tabu Search produced too many configurations once instance groups, repeated runs, and nontrivial time budgets entered the picture. The project therefore moved to a sequential strategy: first fix a reasonable anchor tenure and search for alpha, then fix the best alpha and search for tenure. That cut the computational burden while keeping the trends interpretable.

Alpha itself also caused some confusion at the beginning. The first intuition pushed toward a simple rule: lower alpha should always work better because greedier construction should produce stronger starts. The experiments broke that idea. Once the large instances showed that GRASP plus Tabu Search prefers a very high alpha, the logs and the code forced a return to first principles. That moment led directly to our breadth-versus-depth conjecture.

Reproducibility required more care than expected. Once calibration and comparison split into different scripts, manual bookkeeping became dangerous. The final workflow solved that problem in two ways: paired seeds by run index and automatic reuse of calibrated parameters in the comparison stage.

Three additional implementation difficulties deserve a direct mention because they mattered most in the Tabu Search version. The first one concerned the tabu tenure dilemma. A short tenure intensifies too aggressively, revisits very similar structures, and gets stuck in narrow regions of the search space. A long tenure over-diversifies, blocks too many useful returns, and damages average quality. Tenure therefore became a real calibration problem instead of a casual constant.

The second one came from Python data structures. A tabu list sounds simple on paper, but neighborhood exploration asks again and again whether a candidate move is tabu. Those membership checks had to stay cheap. Several representations entered the discussion, including

circular-list ideas combined with dictionaries or set-like checks. The final code keeps the structure readable, but the implementation never ignored the fact that a slow tabu-membership test could eat a meaningful fraction of the runtime budget.

After evaluating options (list+dict, circular buffer, set), we chose a FIFO Python `list` for the tabu memory and accepted O(tenure) membership tests because practical tenure sizes kept checks cheap; this choice prioritizes readability and easier debugging over micro-optimisation.

The third one returned to time control. Tabu Search does not stop naturally at a local optimum, so stopping near the 10-second limit during calibration or the 30-second limit during the final comparison required explicit checks inside the main while-loop. Because these checks occur at the loop boundary after completing a swap evaluation, the algorithm allows for slight natural overruns (e.g., stopping at 31 seconds) rather than brutally interrupting the execution. Without those checks, the algorithm could overrun the assigned budget and break the fairness of the comparison.

All these difficulties shaped both the code and the interpretation of the experiments. The tenure dilemma clarified why an intermediate memory depth works better than extreme values. The Python data-structure issue showed that runtime-limited experiments depend on implementation efficiency and not only on high-level algorithm design. The time-budget issue reinforced the same lesson from another angle: equal limits only matter if the code really respects them.

5. Calibration of Alpha and Tabu Tenure

The final comparison used all 15 benchmark instances from the MDG repository (6 small and 9 large), while the calibration stage used a representative subset of 10 instances (5 small and 5 large) to keep the computational burden reasonable. Calibration and comparison treat both groups separately because scale changes the role of the parameters, and the data confirmed that expectation.

Calibration mattered because arbitrary parameter choices could distort the final comparison. The two algorithms do not exploit construction quality in the same way, so the baseline GRASP received its own direct calibration, while GRASP plus Tabu Search followed a sequential calibration over alpha and tenure.

Calibration used the alpha set {0.1, 0.25, 0.5, 0.75, 0.9, -1}. For GRASP+TS we followed a sequential calibration: first an alpha sweep with tenure anchored at 15, then a tenure sweep fixing alpha to the best alpha from the first stage. Calibration runs used 10 s per run and 3 repeats per configuration on 10 representative instances (5 small, 5 large). Final chosen settings were: small group GRASP alpha = 0.1, GRASP+TS alpha = 0.25 and tenure = 15; large group GRASP alpha = 0.1, GRASP+TS alpha = 0.9 and tenure = 10.

That choice turned out to fit the problem well. The small instances saturate quickly and mostly work as a consistency check. The large instances preserve enough difficulty for alpha and tenure to change behavior in a visible way.

Table 1. Experimental protocol used in calibration and final comparison.

Protocol item	Calibration stage	Final comparison stage
Time budget per run	10 seconds	30 seconds

Runs per configuration or instance	3	5
Instance set	10 representative instances (5 small, 5 large)	15 benchmark instances (6 small, 9 large)
Reproducibility	Fixed seeds plus CSV/JSON summaries	Fixed paired seeds plus CSV/Excel outputs

Table 2. Calibrated parameter settings selected for each instance group.

Group	Algorithm	Selected alpha	Selected tenure	Best avg dev% in calibration
Small	GRASP	0.1	-	0.0000
Small	GRASP+TS	0.25	15	0.0000
Large	GRASP	0.1	-	0.2536
Large	GRASP+TS	0.9	10	0.0906

The most important calibration result appears on the large instances: the two algorithms move toward opposite alpha values. Plain GRASP prefers alpha = 0.1, while GRASP plus Tabu Search prefers alpha = 0.9 and tenure = 10. The data suggest a structural effect rather than noise. When the improvement phase stays weaker and purely improving, a strong start matters more. When the improvement phase can escape weaker regions, broader and more random starts gain value.

A full joint calibration over alpha and tenure would look more exhaustive in principle, but it would also cost much more time. The sequential strategy therefore acts as a compromise between thoroughness and feasibility. Given the clarity of the final trends, that compromise looks justified.

The small-instance calibration results matter precisely because they stay almost flat. When nearly every parameter value reaches the same objective value with almost no variance, the selected setting should not pretend to reveal a deep truth about the algorithm. In this group, the parameters behave more like practical defaults. The real story starts on the large instances.

The large-group calibration reveals that baseline GRASP loses quality as alpha moves away from the greedy side of the spectrum. That pattern suggests that exhaustive best-improvement local search benefits from entering a good basin quickly. GRASP has no deliberate mechanism for accepting worsening moves, so the quality of the initial construction matters a lot.

GRASP plus Tabu Search moves in the opposite direction, and that result stands out as one of the most interesting parts of the project. If both algorithms had preferred the same alpha, calibration would still matter, but it would reveal much less. The preference for alpha = 0.9 suggests that diversified starts become useful once Tabu Search can intensify and repair them afterwards.

The tenure result also fits the observed behavior. A short tenure makes cycling too easy and weakens memory. A long tenure constrains the search too much and blocks moves that still have value. The selected value of 10 on the large instances gives the best balance between those two extremes.

6. Final Comparison: Did Tabu Search Improve GRASP?

Calibration alone could not answer the main question of the project. Tuned parameters still needed a final test under equal runtime limits and paired seed indices. That stage asked directly whether Tabu Search improves the basic GRASP under the same time budget.

The paired seeds matter because stochastic algorithms can vary a lot from run to run, and comparable random conditions make the contrast cleaner. The Wilcoxon signed-rank test also fits the design because the outcomes are paired and the analysis does not need a normality assumption.

The experimental protocol stayed fixed throughout the final comparison: TIME_LIMIT = 30 seconds per instance, RUNS_PER_INSTANCE = 5, and fixed paired seeds generated from RANDOM_SEED = 42. Calibration was carried out on a representative subset of 10 instances, while the final comparison used all 15 benchmark instances spanning n = 100 and n = 500. Outputs were exported to CSV, JSON, and spreadsheet files for reproducibility.

The small instances give a simple answer: Tabu Search did not improve GRASP because GRASP had already saturated the problem under the available budget. All 30 paired runs ended in exact ties. That result still carries information because it shows that the advanced metaheuristic becomes redundant once the instances turn easy enough.

Table 3. Summary of the 30-second comparison between GRASP and GRASP plus Tabu Search.

Scope	Instances	GRASP Avg Dev (%)	TS Avg Dev (%)	GRASP Mean Std	TS Mean Std	Paired Runs (GRASP / Ties / TS)	Mean Delta (TS-GRASP)	p-value	Reading
Small	6	0.0000	0.0000	0.00	0.00	0 / 30 / 0	0.0000	n.a.	Both methods saturate the instances.
Large	9	0.5200	0.8443	20.57	41.69	30 / 0 / 15	-25.1271	0.002628	GRASP is better on average; TS is more variable.
Overall	15	0.3120	0.5066	12.34	25.01	30 / 30 / 15	-15.0763	0.002628	The idea is interesting, but it does not beat GRASP on average.

[INSERTAR AQUI DIAGRAMA DE BARRAS GENERADO EN EXCEL MOSTRANDO COMPARATIVA DE MEDIAS Y DESVIACIONES DE GRASP FRENTE A GRASP+TS]

[INSERTAR AQUI DIAGRAMA DE BARRAS GENERADO EN EXCEL MOSTRANDO VARIABILIDAD MEDIA Y RECUENTO DE VICTORIAS EMPAREJADAS]

The Wilcoxon test on the 45 paired large-instance runs yields p = 0.002628, supporting that GRASP's average advantage under the 30-second budget is statistically significant.

The large instances show that Tabu Search did not improve GRASP in average quality under the same 30-second budget. The baseline GRASP achieves lower average deviation, lower variance, and 30 wins out of 45 paired runs. The Wilcoxon p-value of 0.002628 supports that difference as statistically significant. If the question asks whether the advanced metaheuristic improves the baseline in average performance under equal time, the answer is no.

The result still does not reduce to a simple claim that Tabu Search failed. A more nuanced pattern appears in the data. GRASP plus Tabu Search reaches the experiment-best value on more large instances than the baseline, and across all 75 paired runs it matches that experiment-best 35 times versus 34 for GRASP. That shows that the stronger improvement phase can still produce excellent peaks. The problem is variance. Under a fixed time budget, that higher variance hurts the average result.

The instance MDG-a_5 shows that pattern clearly. In one run, GRASP plus Tabu Search reaches the best observed value of 7755.23, but several other runs fall far below that level, so the mean deteriorates sharply. MDG-a_13 shows the other side of the story. On MDG-a_13, the mean deviation for GRASP was 0.5463% versus 0.4148% for GRASP+TS, so one of the few large instances actually rewards the deeper search on average. The right interpretation therefore does not look like a universal ranking. Everything depends on whether the goal values stable average quality or occasional high peaks.

That contrast between average quality and peak quality stands out as one of the strongest results in the project. For stable average performance under a fixed budget, GRASP wins in these experiments. For situations that value occasional very high outcomes and can tolerate instability, GRASP plus Tabu Search still offers a real advantage.

The runtime budget also explains part of the result. Thirty seconds favors methods that can complete many competitive restarts. Under that limit, the breadth of GRASP dominates. A longer budget could change the balance because GRASP plus Tabu Search would gain more opportunities to combine deep exploitation with several independent starts. The conclusion therefore belongs to the time limits that the project actually tested.

The paired nonparametric test matters here because a few spectacular runs can distort the informal reading of stochastic results. Wilcoxon checks whether the direction of the differences stays systematic across matched runs, and the significant result on the large group confirms that the average advantage of GRASP is not just an anecdotal impression.

7. Discussion: Our Breadth-versus-Depth Conjecture

Our breadth-versus-depth conjecture explains the results better than any simpler story. Plain GRASP spends less time on each restart and therefore explores many more construction-plus-local-search trajectories. That breadth stabilizes average quality and reduces sensitivity to unlucky starts. GRASP plus Tabu Search does the opposite. It spends more time intensifying each trajectory. That extra depth can produce better peaks, but it also cuts the number of completed restarts under the same budget and raises variance.

The alpha calibration ties directly to the breadth-versus-depth conjecture. Low alpha leads to greedier starts, which suits baseline GRASP because many short restarts remain stable under the 30-second budget. High alpha creates more diverse starts, which GRASP plus Tabu Search can sometimes repair into better peaks; under the short budget, however, that same diversity also comes with higher variance.

Calibration and final comparison both matter for that reason. Calibration prevents a contest between poorly tuned versions of the algorithms. Final comparison prevents a few successful runs from masquerading as proof of superiority. Together, both stages support a conclusion that stays algorithmic and empirical at the same time.

The study also carries clear limitations. The experiments only use the benchmark instances available in the repository, the Tabu Search calibration follows a sequential strategy instead of a full joint search over alpha and tenure, and the comparison reference uses the best value observed in our own runs instead of an external literature optimum. None of those limits erase the conclusions, but all of them define the scope. In particular, the sequential calibration assumes low interaction between alpha and tenure. Strong interaction could move the chosen combination away from the global optimum, although the observed trends look too clear to reverse the main conclusion.

A larger runtime budget could also change the ranking. More time would let GRASP plus Tabu Search complete more intensified restarts, and stronger exploitation might then improve average quality as well. The project therefore does not claim a universal result. It claims a result tied to the tested limits.

The reference value in these experiments is the best value observed across the runs in the comparison, not an external best-known value from the literature. This reference supports a controlled internal comparison.

The current implementation leaves several direct extensions open. Implementing node frequencies in the future could support long-term diversification, a full joint calibration of alpha and tenure could replace the sequential one, and richer neighborhoods could go beyond the basic swap move.

8. Conclusion

Did Tabu Search improve the base GRASP algorithm? Under the 30-second budget tested in this project, the answer is no in average performance. GRASP reached a lower average deviation on the large instances (0.5200% versus 0.8443%), won 30 of the 45 large paired runs, and the Wilcoxon p-value of 0.002628 confirms that this average advantage is statistically significant. On the small instances, both methods tied in all 30 paired runs, so Tabu Search added no practical benefit there.

The runtime budget provides a computational explanation for this outcome. The baseline GRASP is fast enough to complete many competitive restarts within 30 seconds, so it repeatedly falls into high-quality basins and stabilizes its average result. GRASP plus Tabu Search spends more of that same budget on intensification, tabu checks, and longer trajectories; that added depth can produce stronger peaks in selected runs, but it also reduces the number of independent restarts and worsens the mean under a short budget.